

Score-Based Diffusion Model

An introduction

Yukinari Hisaki

Contents

1. Introduction
2. Why Diffusion?
3. Score Matching Basics
4. Diffusion as an SDE
5. Generation via Probability Flow ODE
6. Specify the Forward Process: VP SDE
7. Learn the Time-dependent Score
8. Controllable Generation: Guiding the SDE

- 9.** Advanced Capabilities Enabled by the ODE Formulation
- 10.** Accelerating Generation: DPM-Solver++

Introduction

- The purpose of this talk is to understand the Diffusion Model, which is part of the Diffusion Planner algorithm.
- Diffusion Model is more difficult than previous talk about Transformer. Diffusion Model is based on the difficult mathematical theory.
- However, the outcome of this theory is a simple algorithm.
- A rough understanding of the theory is sufficient; you don't need to worry about the details.

Why Diffusion?

The central goal is to estimate an unknown data distribution $q(x)$ from samples $x \sim q(x)$.

- **The Standard Approach (e.g., VAEs):** Introduce a latent variable $z \sim \mathcal{N}(0, I)$ and map it to data via a **single** probabilistic decoding step $p_{\theta}(x|z)$.

$$z \xrightarrow{p_{\theta}(x|z)} x$$

- **The Limitation:** Learning the complex data distribution $q(x)$ via a “**single shot**” $p_{\theta}(x|z)$ is challenging. The gap between the simple prior and the complex data manifold is too large.
- **The Solution (Diffusion Models):** Decompose the generation into **multiple gradual transformations**. We refine the sample step-by-step. $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$

$$x_T \sim \mathcal{N}(\cdot \mid 0, I), x_0 \sim q(x_0)$$

The idea of “gradual transformation” can be mathematically formulated in three main ways:

1. Denoising Diffusion Probabilistic Models (DDPM) [1]

- Formulated as discrete Markov chains.
- Optimized via Variational Lower Bound (ELBO).
- Key idea: Learn to reverse the noise addition step-by-step.

2. Score-based Generative Models [2]

- Formulated via Stochastic Differential Equations (SDEs).
- Key idea: Learn the **score function** $\nabla_x \log p_t(x)$ (direction to higher density).

3. Flow Matching

- A more general framework based on Ordinary Differential Equations (ODEs).
- Key idea: Directly regress a **vector field** $v_t(x)$ to transport probability paths.

Focus of this talk: We will focus on **2. Score Matching**.

Score Matching Basics

Before introducing diffusion, let's define the “Score”.

Definition 0.1 (Score): For a probability density $q(x)$, the score is defined as the gradient of the log-density with respect to data x :

$$s_t(x) := \nabla_x \log p_t(x).$$

- **Intuition: A Vector Field**

- ▶ The score is a vector field pointing in the direction of **higher data density**.
- ▶ At the modes (peaks) of the distribution, the score is zero ($\nabla \log q(x) = 0$).

- **The Goal (Score Matching)** We want to train a neural network $s_{\theta(x)}$ to approximate this true score:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{x \sim q} [\| s_{\theta}(x) - \nabla_x \log q(x) \|_2^2]$$

Problem: The true score $\nabla_x \log q(x)$ is intractable.

Theorem 0.1 (Equivalence of Explicit and Denoising Score Matching): Let $q(x)$ be the data distribution and $q(\tilde{x}|x)$ be a perturbation kernel. Define the smoothed marginal distribution as $q(\tilde{x}) = \int q(\tilde{x}|x)q(x) dx$.

Optimizing the explicit score matching objective for the smoothed marginal,

$$\mathcal{L}_{\text{ESM}}(\theta) = \mathbb{E}_{q(\tilde{x})} [\| s_{\theta(\tilde{x})} - \nabla_{\tilde{x}} \log q(\tilde{x}) \|_2^2],$$

is equivalent to optimizing the Denoising Score Matching objective up to a constant C independent of θ :

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{q(x)q(\tilde{x}|x)} [\| s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q(\tilde{x}|x) \|_2^2].$$

$$\Rightarrow \mathcal{L}_{\text{ESM}}(\theta) = \mathcal{L}_{\text{DSM}}(\theta) + C$$

The Core Intuition: Calculating the true score $\nabla_{\tilde{x}} \log q(\tilde{x})$ is computationally intractable. This theorem bypasses the problem by matching the score of $q(\tilde{x}|x)$ instead. Since we explicitly design this perturbation kernel, its score is fully known. We effectively replace an impossible calculation with a tractable, human-defined one.

Choice of Conditional Distribution: Typically, we choose a **Gaussian with small variance** for $q(\tilde{x}|x)$ (e.g., $\mathcal{N}(\tilde{x}; x, \sigma^2 I)$). This makes the conditional score analytically tractable:

$$\nabla_{\tilde{x}} \log q(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2}$$

and allows $q(\tilde{x})$ to approximate the data distribution $q(x)$ closely.

Once we have the score function $s_{\theta(x)} \approx \nabla_x \log q(x)$, how do we generate samples?

- **Langevin Dynamics:** We initialize x_0 arbitrarily and iteratively update it using the score:

$$x_{t+1} \leftarrow x_t + \underbrace{\varepsilon_t s_{\theta}(x_t)}_{\text{Gradient Ascent}} + \underbrace{\sqrt{2\varepsilon_t} z_t}_{\text{Random Noise}}$$

where $z_t \sim \mathcal{N}(0, I)$ and ε is the step size.

- **Mechanism:**
 1. **Gradient Term:** Pushes x towards high-density regions (modes).
 2. **Noise Term:** Prevents collapsing to a single mode, allowing exploration of the full distribution.

$\Rightarrow$ As $t \rightarrow \infty$ and $\varepsilon_t \rightarrow 0$, x_t converges to a sample from $q(x)$.

Theory is sound, but training on raw data $x \sim q(x)$ often fails in practice.

1. The Manifold Hypothesis (Blindness)

- High-dimensional data concentrates on a low-dimensional manifold.
- Outside this manifold, $q(x) \approx 0$.
- The score $\nabla_x \log q(x)$ is **undefined** or numerically unstable in the ambient space.

2. Inaccurate Estimation in Low-Density Regions

- The loss $\mathbb{E}_{x \sim q}$ only minimizes error **where data exists**.
- In low-density regions (most of the space), the model $s_{\theta(x)}$ is untrained.
- **Result:** When sampling starts from random noise (low density), the gradients are inaccurate, leading to garbage generation.

3. Slow Mixing of Langevin Dynamics

- If the data distribution has disjoint modes (isolated “islands” of data), the score in between is near zero.
- The sampler gets stuck in one mode and cannot cross the gap to others.

Diffusion as an SDE

How do we populate low-density regions and fix the undefined score?

Solution: Continuously perturb the data into pure noise over time $t \in [0, T]$.

Theorem 0.2 (Forward SDE and its marginal evolution): Assume the **forward** diffusion is defined by the Itô SDE:

$$dx = f(x, t) dt + g(t) dw, \quad t \in [0, T],$$

where w is a standard Wiener process, $f(x, t)$ is the **drift** coefficient, and $g(t)$ is the scalar **diffusion** coefficient.

The marginal density $p_t(x)$ exists and evolves according to the **Fokker-Planck PDE**:

$$\partial_t p_t(x) = -\nabla \cdot (f(x, t)p_t(x)) + \frac{1}{2}g(t)^2 \Delta p_t(x).$$

- **What this Theorem Means:** Simply put, if we take a data point sampled from $x_0 \sim p_0(x)$ and move it according to the SDE, this PDE describes exactly what its probability distribution $x_t \sim p_t(x)$ looks like at any time t .
 - **Designing the SDE:** We design the drift $f(x, t)$ and diffusion $g(t)$ specifically so that as $t \rightarrow T$, the distribution $p_t(x)$ smoothly converges to a simple, tractable noise distribution. For example, setting $f(x, t) = -\frac{1}{2}\beta(t)x$ and $g(t) = \sqrt{\beta(t)}$ ensures $p_T(x) \approx \mathcal{N}(0, I)$.
 - **The Continuous Perturbation Process:**
 - **At $t = 0$:** $x_0 \sim p_0(x)$ (The complex, low-dimensional real data distribution).
 - **At $t \in (0, T)$:** Noise gradually fills the empty space. The smoothed score $\nabla_x \log p_t(x)$ becomes well-defined everywhere.
 - **At $t = T$:** $x_T \sim p_T(x) \approx \mathcal{N}(0, I)$ (An unstructured, tractable prior).
- ⇒ **Key Idea:** By designing $f(x, t)$ and $g(t)$, we construct a process that systematically destroys data into noise. If we can **reverse** this SDE, we can generate data from noise!

Generation via Probability Flow ODE

The forward SDE transforms data $p_0(x)$ into pure noise $p_T(x) \approx \mathcal{N}(0, I)$. To generate data, we reverse this process from $t = T$ down to $t = 0$. Remarkably, we can perfectly retrace the exact marginal densities $p_t(x)$ using a deterministic equation.

Theorem 0.3 (Probability Flow ODE (Song et al., 2020)): For the forward SDE $dx = f(x, t) dt + g(t) dw$, there exists a corresponding deterministic **Ordinary Differential Equation (ODE)**. This ODE shares the **exact same marginal probability densities** $p_t(x)$ at all times $t \in [0, T]$:

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt.$$

- **The Missing Piece:** To simulate this ODE backwards from $t = T$ to $t = 0$, $f(x, t)$ and $g(t)$ are already known (we designed them). The **only** unknown variable is the score function $\nabla_x \log p_t(x)$. $\Rightarrow$ **Our trained score model $s_\theta(x, t)$ directly plugs in here!**

Let's pause and summarize the theoretical framework we have established.

- **The Key Takeaway:** Given a forward SDE $dx = f(x, t) dt + g(t) dw$, there exists a corresponding Probability Flow ODE:

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt$$

The mathematical blueprint to generate data is entirely determined by the forward process and its score.

- **What remains to be done?** To turn this continuous-time theory into a practical generative model, we must complete two concrete tasks:
 - 1. Design a Specific Forward SDE:** We need to explicitly define the drift $f(x, t)$ and diffusion $g(t)$ coefficients so that the complex data $p_0(x)$ reliably converges to a tractable noise prior $p_T(x) \approx \mathcal{N}(0, I)$.
 - 2. Learn the Score Function:** Since the probability flow ODE requires $\nabla_x \log p_t(x)$, we need a way to train a time-dependent neural network $s_{\theta(x, t)}$ to approximate this score for **all** continuous times $t \in [0, T]$.
- ⇒ Let's first address **Task 1** by looking at a standard choice: the **Variance Preserving (VP) SDE**.

Specify the Forward Process: VP SDE

A standard choice for the forward process is the **Variance Preserving (VP) SDE**, which is the continuous-time limit of DDPM.

- **The SDE Formulation:** We define the drift and diffusion using a time-dependent noise schedule $\beta(t) > 0$:

$$dx = \underbrace{-\frac{1}{2}\beta(t)x}_{\text{Drift: Shrinks signal}} dt + \underbrace{\sqrt{\beta(t)}}_{\text{Diffusion: Injects noise}} dw$$

- **Why “Variance Preserving”?** If the initial data x_0 has unit variance, the variance of x_t remains exactly 1 for all t . The drift smoothly scales down the data toward the origin, while the diffusion perfectly compensates by adding Gaussian noise.
- **The Tractable Transition Kernel:** Because this SDE is linear, the transition probability from x_0 to x_t is always a tractable Gaussian. Let $\alpha_t = \exp\left(-\int_0^t \beta(s) ds\right)$:

$$p_{t|0}(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$$

- **Practical Implementation (Linear Schedule):** In practice, we set $t \in [0, 1]$ and define $\beta(t)$ as a linear function:

$$\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min})$$

Standard hyperparameters are $\beta_{\min} = 0.1$ and $\beta_{\max} = 20$.

This gives a simple, closed-form solution for code implementation:

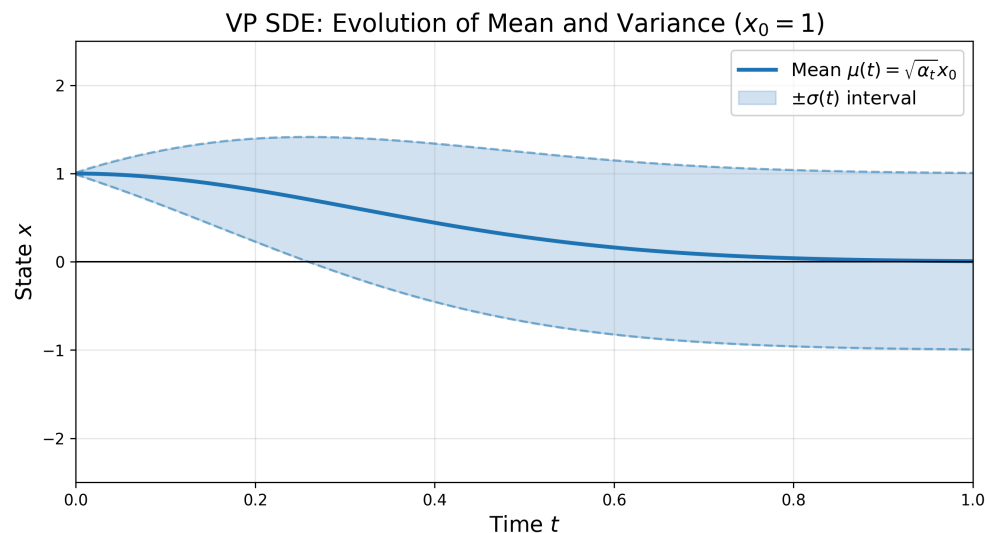
$$\begin{aligned} \int_0^t \beta(s) \, ds &= \beta_{\min} t + \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2 \\ \Rightarrow \alpha_t &= \exp\left(-\beta_{\min} t - \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2\right) \end{aligned}$$

$\Rightarrow$ At $t = 1$, $\alpha_1 \approx \exp(-10) \approx 4.5 \times 10^{-5} \approx 0$. The signal is completely destroyed, ensuring $p_1(x) \approx \mathcal{N}(0, I)$.

- **The Corresponding Probability Flow ODE:** By substituting our specific drift $f(x, t)$ and diffusion $g(t)$ into the general Probability Flow ODE formula, we get the following equation:

$$dx = \left(-\frac{1}{2}\beta(t)x - \frac{1}{2}\beta(t)\nabla_x \log p_t(x) \right) dt$$

- **Visualizing the Evolution ($x_0 = 1$):** How do these equations transform our data? Let's track a single 1D data point starting exactly at $x_0 = 1$ over time $t \in [0, 1]$.



- **Drift (The Mean):** The mean $\mu(t) = \sqrt{\alpha_t} x_0$ smoothly decays from 1 down to 0. The original signal is systematically erased.
- **Diffusion (The Variance):** The $\pm 2\sigma(t)$ interval, where $\sigma(t) = \sqrt{1 - \alpha_t}$, steadily expands until the distribution perfectly matches standard Gaussian noise $\mathcal{N}(0, 1)$ at $t = 1$.

Learn the Time-dependent Score

We now have a specific SDE and its transition kernel. The final task is to train a neural network $s_{\theta(x,t)}$ to approximate the score $\nabla_x \log p_t(x)$ across all continuous times $t \in [0, 1]$.

- **Extending Denoising Score Matching:** Just like in the discrete case, matching the intractable marginal score is mathematically equivalent to matching the **known** conditional score $\nabla_{x_t} \log p_{t|0}(x_t \mid x_0)$.
- **The Conditional Score of the VP SDE:** Since the transition kernel is $p_{t|0}(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$, the score is simply the gradient of the Gaussian log-density:

$$\nabla_{x_t} \log p_{t|0}(x_t \mid x_0) = -\frac{x_t - \sqrt{\alpha_t}x_0}{1 - \alpha_t}$$

Notice: This is a deterministic, closed-form target!

- **The Continuous-Time Objective:** We train our score network $s_{\theta(x_t, t)}$ by minimizing the expected distance to this conditional score. We sample time t uniformly, original data x_0 , and perturbed data x_t .

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1)} \mathbb{E}_{x_0 \sim p_0} \mathbb{E}_{x_t \sim p_{t|0}} \left[\lambda(t) \left\| s_{\theta(x_t, t)} - \underbrace{\left(-\frac{x_t - \sqrt{\alpha_t} x_0}{1 - \alpha_t} \right)}_{\nabla_{x_t} \log p_{t|0}(x_t | x_0)} \right\|_2^2 \right]$$

- **The Weighting Function $\lambda(t)$:** The score magnitude grows extremely large as $t \rightarrow 0$ (because the variance $(1 - \alpha_t) \rightarrow 0$). To prevent gradient explosion and balance the loss across all times, we typically choose a positive weighting function proportional to the variance, such as $\lambda(t) = 1 - \alpha_t$.

⇒ By optimizing this single objective, $s_{\theta}(x, t)$ learns the continuous vector field. We can then plug it directly into the Reverse ODE to generate data!

Controllable Generation: Guiding the SDE

We now know how to generate unconditional data $x \sim p_0(x)$. What if we want to generate data conditioned on a signal y (e.g., a text prompt or a class label)?

- **The Conditional Score:** To generate conditional samples, we simply replace our unconditional score $\nabla_x \log p_t(x)$ with the conditional score $\nabla_x \log p_t(x | y)$ inside the Reverse ODE/SDE.
- **Applying Bayes' Theorem:** By taking the gradient of the log-posterior $p_t(x | y) \propto p_t(x)p_t(y | x)$, we get a beautiful decomposition:

$$\nabla_x \log p_t(x | y) = \underbrace{\nabla_x \log p_t(x)}_{\substack{\text{Unconditional Score} \\ s_\theta(x,t)}} + \underbrace{\nabla_x \log p_t(y | x)}_{\substack{\text{Guidance Term} \\ \text{(Likelihood)}}}$$

- **The Mechanism:**
 1. **Unconditional Score:** Ensures the generated sample looks like real, high-quality data.
 2. **Guidance Term:** Acts as a secondary vector field, actively pushing the trajectory toward regions of the data manifold that satisfy the condition y .

In **Diffusion Planner**, we want to generate valid trajectories that satisfy specific physical or operational constraints (e.g., avoiding obstacles, target speeds).

- **Rethinking the Guidance Term:** Instead of training a separate neural network classifier to predict $p_t(y \mid x)$, we can mathematically define these rules using **human-designed cost functions** $\mathcal{E}(x)$.
- **Energy-Based Formulation:** We formulate the condition y (“satisfies all constraints”) as a Boltzmann distribution. The probability of a trajectory being valid decreases exponentially as the cost increases:

$$p_t(y \mid x_t) \propto \exp(-\lambda \mathcal{E}(x_t))$$

where $\lambda > 0$ controls the guidance strength, and $\mathcal{E}(x_t)$ is the total cost.

- **Analytic Guidance Field:** By plugging this into our Bayes' decomposition, the intractable log-probability derivative beautifully simplifies into an **analytic gradient**:

$$\nabla_{x_t} \log p_t(y \mid x_t) = -\lambda \nabla_{x_t} \mathcal{E}(x_t)$$

(e.g., $\mathcal{E}(x_t) = \mathcal{E}_{\text{obstacle}}(x_t) + \mathcal{E}_{\text{velocity}}(x_t)$)

⇒ **The Synergy:** The unconditional score $s_\theta(x_t, t)$ ensures the trajectory follows realistic dynamics (learned from data), while $-\lambda \nabla_{x_t} \mathcal{E}(x_t)$ acts as a “repulsive force” pushing the path away from obstacles during inference!

Advanced Capabilities Enabled by the ODE Formulation

Because the Probability Flow ODE is completely **deterministic**, it defines a bijective (1-to-1) mapping between the complex data space $x_0 \sim p_0$ and the simple latent space $x_T \sim p_T$.

- **How to Interpolate:**

1. **Encode:** Take two real images $x_0^{(A)}$ and $x_0^{(B)}$. Simulate the ODE **forward** ($t = 0 \rightarrow T$) to find their exact latent representations $x_T^{(A)}$ and $x_T^{(B)}$.
 2. **Interpolate:** Blend the latents in the spherical prior space using a parameter $\lambda \in [0, 1]$ (e.g., $z_\lambda = \text{Slerp}(x_T^{(A)}, x_T^{(B)}, \lambda)$).
 3. **Decode:** Simulate the ODE **backward** ($t = T \rightarrow 0$) starting from z_λ to generate the intermediate image $x_0^{(\lambda)}$.
- **Result:** Unlike pixel-space blending, ODE-based interpolation smoothly morphs the semantic features (e.g., age, hair color) while staying perfectly on the data manifold.

Application 1: Latent Space Interpolation

35 / 41



The Reverse ODE allows us to compute the **exact** probability density $p_0(x)$ of any data point.

- **Instantaneous Change of Variables:** Let $f_{\text{ODE}}(x, t)$ be the vector field of our Probability Flow ODE. The change in the log-probability over time is given by the negative divergence of this vector field:

$$\frac{\partial \log p_t(x)}{\partial t} = -\nabla_x \cdot f_{\text{ODE}}(x, t)$$

- **Computing the Likelihood:** To find the exact log-likelihood of a real image x_0 , we integrate this divergence along the ODE trajectory from $t = 0$ to $t = T$:

$$\log p_0(x_0) = \log p_T(x_T) + \int_0^T \nabla_{x_t} \cdot f_{\text{ODE}}(x_t, t) dt$$

where $p_T(x_T)$ is just the simple Gaussian prior $\mathcal{N}(0, I)$.

- **Why this matters:** Exact likelihoods allow for principled model evaluation, rigorous comparison against other generative models (like Autoregressive models or Normalizing Flows), and out-of-distribution (anomaly) detection.

Accelerating Generation: DPM-Solver++

While the Probability Flow ODE is elegant, solving it with standard numerical methods (like Euler or Runge-Kutta) is notoriously slow, often requiring 200 to 1000 steps.

- **Why so slow?** The ODE contains a “stiff” linear drift term. If we take large step sizes, the truncation error explodes, destroying the image.

$$dx = \underbrace{-\frac{1}{2}\beta(t)x}_{\text{Linear Drift (Stiff)}} dt - \frac{1}{2}\beta(t) \underbrace{\varepsilon_{\theta}(x, t)}_{\text{Neural Network}} dt$$

- **The Semi-Analytic Solution (DPM-Solver):** Instead of treating the entire ODE as a black box, Lu et al. (2022) proposed using an **Exponential Integrator**. By applying the variation of constants formula, DPM-Solver solves the linear drift part $f(x, t)$ **exactly analytically**. It only relies on numerical approximation for the non-linear neural network part $\varepsilon_{\theta}(x, t)$.

⇒ This reduces the required steps from 1000 down to around 50. But can we go even faster?

DPM-Solver is fast, but it suffers from instability at very small t (near the end of generation), especially when using large Classifier-Free Guidance scales. **DPM-Solver++** fixes this via a clever reparameterization.

- **Noise vs. Data Prediction:** Instead of approximating the noise trajectory $\varepsilon_{\theta(x,t)}$, DPM-Solver++ rewrites the exact ODE solution to integrate over the **data prediction** model $x_{\theta(x,t)} \approx x_0$:

$$x_{\theta(x_t,t)} = \frac{x_t - \sqrt{1 - \alpha_t} \varepsilon_{\theta(x_t,t)}}{\sqrt{\alpha_t}}$$

- **Why predict the clean image x_0 ?** The predicted noise ε oscillates sharply as the generation progresses. However, the model's prediction of the final clean image x_θ becomes remarkably **stable and smooth** as t decreases. Integrating a mathematically **smoother function** allows for Taylor expansions with much larger step sizes and lower truncation error!
- **The Impact:** By applying a multi-step update rule on this smooth x_θ trajectory, DPM-Solver++ can generate extremely high-quality images in just **10 to 20 steps**, effectively making real-time diffusion a reality.

References

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
- [2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” in International Conference on Learning Representations (ICLR), 2021. [Online]. Available: <https://openreview.net/forum?id=PxTIG12RRHS>
- [3] Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” in Advances in Neural Information Processing Sys-

tems (NeurIPS), 2019. [Online]. Available: <https://papers.nips.cc/paper/9361-generative-modeling-by-estimating-gradients-of-the-data-distribution>